Name - Vedant Satish Gunjal
Roll No- 407I083
Class - BEIT
Title - Use Autoencoder to implement anomaly detection.
Build the model by using:
a. Import required libraries
b. Upload / access the dataset
c. Encoder converts it into latent representation
d. Decoder networks convert it back to the original input
e. Compile the models with Optimizer, Loss, and Evaluation Metrics

```
import pandas as pd
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler
from sklearn.metrics import confusion_matrix, recall_score, accuracy_score, precision_
score

RANDOM_SEED = 2021
TEST_PCT = 0.3
LABELS = ["Normal","Fraud"]
```

```
dataset = pd.read_csv("creditcard.csv")
```

```
#check for any null values
print("Any nulls in the dataset",dataset.isnull().values.any())
print('-------')
print("No. of unique labels",len(dataset['Class'].unique()))
print("Label values",dataset.Class.unique())

#0 is for normal credit card transcation
#1 is for fraudulent credit card transcation
print('-------')
print("Break down of Normal and Fraud Transcations")
print(dataset['Class'].value_counts(sort=True))
```

```
Any nulls in the dataset False
-------
No. of unique labels 2
Label values [0 1]
-------
Break down of Normal and Fraud Transcations
Class
0    284315
1       492
Name: count, dtype: int64
```

```
#visualizing the imbalanced dataset
count_classes = dataset['Class'].value_counts(sort=True)
count_classes.plot(kind='bar',rot=0)
plt.xticks(range(len(dataset['Class'].unique())),dataset.Class.unique())
plt.title("Frequency by observation number")
plt.xlabel("Class")
plt.ylabel("Number of Observations")

normal_dataset = dataset[dataset.Class == 0]
fraud_dataset = dataset[dataset.Class == 1]
```

```
#Save the normal and fradulent transcations in seperate dataframe
normal_dataset = dataset[dataset.Class == 0]
fraud_dataset = dataset[dataset.Class == 1]

#Visualize transcation amounts for normal and fraudulent transcations
bins = np.linspace(200,2500,100)
plt.hist(normal_dataset.Amount,bins=bins,alpha=1,density=True,label='Normal')
plt.hist(fraud_dataset.Amount,bins=bins,alpha=0.5,density=True,label='Fraud')
plt.legend(loc='upper right')
plt.title("Transcation Amount vs Percentage of Transcaions")
plt.xlabel("Transcation Amount (USD)")
plt.ylabel("Percentage of Transcations")
plt.show()
```

```
C:\Users\AJINKYA\AppData\Local\Temp\ipykernel_18688\2237239387.py:13: User
Warning: FigureCanvasAgg is non-interactive, and thus cannot be shown
  plt.show()
```

```
dataset
```

	Time	V1	V2	V3	V4	V5	V6	
0	0.0	-1.359807	-0.072781	2.536347	1.378155	-0.338321	0.462388	0.
1	0.0	1.191857	0.266151	0.166480	0.448154	0.060018	-0.082361	-0.
2	1.0	-1.358354	-1.340163	1.773209	0.379780	-0.503198	1.800499	0.
3	1.0	-0.966272	-0.185226	1.792993	-0.863291	-0.010309	1.247203	0.
4	2.0	-1.158233	0.877737	1.548718	0.403034	-0.407193	0.095921	0.
...	...	...	...	...	...	...	...	
284802	172786.0	-11.881118	10.071785	-9.834783	-2.066656	-5.364473	-2.606837	-4.
284803	172787.0	-0.732789	-0.055080	2.035030	-0.738589	0.868229	1.058415	0.
284804	172788.0	1.919565	-0.301254	-3.249640	-0.557828	2.630515	3.031260	-0.
284805	172788.0	-0.240440	0.530483	0.702510	0.689799	-0.377961	0.623708	-0.
284806	172792.0	-0.533413	-0.189733	0.703337	-0.506271	-0.012546	-0.649617	1.

284807 rows × 31 columns

```
sc = StandardScaler()
dataset['Time'] = sc.fit_transform(dataset['Time'].values.reshape(-1,1))
dataset['Amount'] = sc.fit_transform(dataset['Amount'].values.reshape(-1,1))
```

```
raw_data = dataset.values
#The last element contains if the transcation is normal which is represented by 0 and
if fraud then 1
labels = raw_data[:,-1]

#The other data points are the electrocadriogram data
data = raw_data[:,0:-1]

train_data,test_data,train_labels,test_labels = train_test_split(data,labels,test_size
= 0.2,random_state =2021)
```

```
# Scale every feature to [0,1] using the min/max of the NORMAL training rows only
normal_rows = train_labels == 0
min_val = train_data[normal_rows].min(axis=0)
max_val = train_data[normal_rows].max(axis=0)
value_range = np.where(max_val - min_val == 0, 1, max_val - min_val)

train_data = ((train_data - min_val) / value_range).astype(np.float32)
test_data = ((test_data - min_val) / value_range).astype(np.float32)
```

```
train_labels = train_labels.astype(bool)
test_labels = test_labels.astype(bool)

#Creating normal and fraud datasets
normal_train_data = train_data[~train_labels]
normal_test_data = test_data[~test_labels]

fraud_train_data = train_data[train_labels]
fraud_test_data = test_data[test_labels]
print("No. of records in Fraud Train Data=",len(fraud_train_data))
print("No. of records in Normal Train Data=",len(normal_train_data))
print("No. of records in Fraud Test Data=",len(fraud_test_data))
print("No. of records in Normal Test Data=",len(normal_test_data))
```

```
No. of records in Fraud Train Data= 389
No. of records in Normal Train Data= 227456
No. of records in Fraud Test Data= 103
No. of records in Normal Test Data= 56859
```

```
nb_epoch = 50
batch_size = 64
input_dim = normal_train_data.shape[1]
#num of columns,30
encoding_dim = 14
hidden_dim1 = int(encoding_dim / 2)
hidden_dim2 = 4
learning_rate = 1e-7
```

```
#input layer
# Build models
input_layer = tf.keras.layers.Input(shape=(input_dim,))



#Encoder
encoder = tf.keras.layers.Dense(encoding_dim,activation="tanh",activity_regularizer =
tf.keras.regularizers.l2(learning_rate))(input_layer)
encoder = tf.keras.layers.Dropout(0.2)(encoder)
encoder = tf.keras.layers.Dense(hidden_dim1,activation='relu')(encoder)
encoder = tf.keras.layers.Dense(hidden_dim2,activation=tf.nn.leaky_relu)(encoder)

#Decoder
decoder = tf.keras.layers.Dense(hidden_dim1,activation='relu')(encoder)
decoder = tf.keras.layers.Dropout(0.2)(decoder)
decoder = tf.keras.layers.Dense(encoding_dim,activation='relu')(decoder)
decoder = tf.keras.layers.Dense(input_dim,activation='tanh')(decoder)

#Autoencoder
# combine encoder and decoder to create the autoencoder model
autoencoder = tf.keras.Model(inputs = input_layer,outputs = decoder)
autoencoder.summary()
```

Model: "functional"

Layer (type)	Output Shape	Param #
input_layer (InputLayer)	(None, 30)	0
dense (Dense)	(None, 14)	434
dropout (Dropout)	(None, 14)	0
dense_1 (Dense)	(None, 7)	105
dense_2 (Dense)	(None, 4)	32
dense_3 (Dense)	(None, 7)	35
dropout_1 (Dropout)	(None, 7)	0
dense_4 (Dense)	(None, 14)	112
dense_5 (Dense)	(None, 30)	450

Total params: 1,168 (4.56 KB)

Trainable params: 1,168 (4.56 KB)

Non-trainable params: 0 (0.00 B)

```
cp = tf.keras.callbacks.ModelCheckpoint(filepath="autoencoder_fraud.h5",mode='min',mon
itor='val_loss',verbose=2,save_best_only=True)
#Define our early stopping
early_stop = tf.keras.callbacks.EarlyStopping(
                monitor='val_loss',
                min_delta=0.0001,
                patience=10,
                verbose=11,
                mode='min',
                restore_best_weights=True
)
```

```
autoencoder.compile(metrics=['accuracy'],loss= 'mean_squared_error',optimizer='adam')
```

```
history = autoencoder.fit(normal_train_data,normal_train_data,epochs = nb_epoch,
                          batch_size = batch_size,shuffle = True,
                          validation_data = (test_data,test_data),
                          verbose=1,
                          callbacks = [cp,early_stop]).history
```

```
Epoch 1/50

Epoch 1: val_loss improved from None to 0.00205, saving model to autoencod
er_fraud.h5

Epoch 1: finished saving model to autoencoder_fraud.h5

3554/3554 ━━━━━━━━━━━━━━━━━━━━ 9s 2ms/step - accuracy: 0.9195 - loss: 0.00
81 - val_accuracy: 0.9521 - val_loss: 0.0020

Epoch 2/50

Epoch 2: val_loss did not improve from 0.00205

3554/3554 ━━━━━━━━━━━━━━━━━━━━ 6s 2ms/step - accuracy: 0.9529 - loss: 0.00
21 - val_accuracy: 0.9521 - val_loss: 0.0021

Epoch 3/50

Epoch 3: val_loss improved from 0.00205 to 0.00203, saving model to autoen
coder_fraud.h5

Epoch 3: finished saving model to autoencoder_fraud.h5

3554/3554 ━━━━━━━━━━━━━━━━━━━━ 7s 2ms/step - accuracy: 0.9522 - loss: 0.00
19 - val_accuracy: 0.9521 - val_loss: 0.0020

Epoch 4/50

Epoch 4: val_loss did not improve from 0.00203

3554/3554 ━━━━━━━━━━━━━━━━━━━━ 7s 2ms/step - accuracy: 0.9524 - loss: 0.00
18 - val_accuracy: 0.9521 - val_loss: 0.0020

Epoch 5/50

Epoch 5: val_loss did not improve from 0.00203

3554/3554 ━━━━━━━━━━━━━━━━━━━━ 7s 2ms/step - accuracy: 0.9529 - loss: 0.00
17 - val_accuracy: 0.9521 - val_loss: 0.0022

Epoch 6/50

Epoch 6: val_loss did not improve from 0.00203

3554/3554 ━━━━━━━━━━━━━━━━━━━━ 8s 2ms/step - accuracy: 0.9531 - loss: 0.00
17 - val_accuracy: 0.9521 - val_loss: 0.0021

Epoch 7/50

Epoch 7: val_loss did not improve from 0.00203

3554/3554 ━━━━━━━━━━━━━━━━━━━━ 8s 2ms/step - accuracy: 0.9531 - loss: 0.00
17 - val_accuracy: 0.9521 - val_loss: 0.0021

Epoch 8/50

Epoch 8: val_loss did not improve from 0.00203

3554/3554 ━━━━━━━━━━━━━━━━━━━━ 8s 2ms/step - accuracy: 0.9531 - loss: 0.00
17 - val_accuracy: 0.9521 - val_loss: 0.0022
```

```
Epoch 9/50

Epoch 9: val_loss did not improve from 0.00203

3554/3554 ━━━━━━━━━━━━━━━━━━━━ 8s 2ms/step - accuracy: 0.9531 - loss: 0.00
17 - val_accuracy: 0.9521 - val_loss: 0.0021

Epoch 10/50

Epoch 10: val_loss did not improve from 0.00203

3554/3554 ━━━━━━━━━━━━━━━━━━━━ 8s 2ms/step - accuracy: 0.9531 - loss: 0.00
16 - val_accuracy: 0.9521 - val_loss: 0.0021

Epoch 11/50

Epoch 11: val_loss did not improve from 0.00203

3554/3554 ━━━━━━━━━━━━━━━━━━━━ 7s 2ms/step - accuracy: 0.9530 - loss: 0.00
16 - val_accuracy: 0.9521 - val_loss: 0.0021

Epoch 11: early stopping

Restoring model weights from the end of the best epoch: 1.
```

```
plt.plot(history['loss'],linewidth = 2,label = 'Train')
plt.plot(history['val_loss'],linewidth = 2,label = 'Test')
plt.legend(loc='upper right')
plt.title('Model Loss')
plt.ylabel('Loss')
plt.xlabel('Epoch')

#plt.ylim(ymin=0.70,ymax=1)

plt.show()
```

```
C:\Users\AJINKYA\AppData\Local\Temp\ipykernel_18688\1376279493.py:10: User
Warning: FigureCanvasAgg is non-interactive, and thus cannot be shown
  plt.show()
```

```
test_x_predictions = autoencoder.predict(test_data)
mse = np.mean(np.power(test_data - test_x_predictions, 2),axis = 1)
error_df = pd.DataFrame({'Reconstruction_error':mse,
                         'True_class':test_labels})
```

```
1781/1781 ━━━━━━━━━━━━━━━━━━━━ 0s 755us/step

1781/1781 ━━━━━━━━━━━━━━━━━━━━ 1s 764us/step
```

```
# Pick the threshold from the reconstruction error of NORMAL training data (99th perce
ntile)
train_pred = autoencoder.predict(normal_train_data)
train_mse = np.mean(np.power(normal_train_data - train_pred, 2), axis=1)
threshold_fixed = np.percentile(train_mse, 99)
print("Threshold =", threshold_fixed)

groups = error_df.groupby('True_class')
fig,ax = plt.subplots()

for name,group in groups:
        ax.plot(group.index,group.Reconstruction_error,marker='o',ms = 3.5,linestyle
='',
                label = "Fraud" if  name==1 else "Normal")
ax.hlines(threshold_fixed,ax.get_xlim()[0],ax.get_xlim()[1],colors="r",zorder=100,labe
l="Threshold")
ax.legend()
plt.title("Reconstructions error for normal and fraud data")
plt.ylabel("Reconstruction error")
plt.xlabel("Data point index")
plt.show()
```

```
7108/7108 ━━━━━━━━━━━━━━━━━━━━ 5s 755us/step

Threshold = 0.00642968

C:\Users\AJINKYA\AppData\Local\Temp\ipykernel_18688\1568287753.py:18: User
Warning: FigureCanvasAgg is non-interactive, and thus cannot be shown
  plt.show()
```

```
pred_y = [1 if e > threshold_fixed else 0
          for e in
        error_df.Reconstruction_error.values]
error_df['pred'] = pred_y
conf_matrix = confusion_matrix(error_df.True_class,pred_y)

plt.figure(figsize = (4,4))
sns.heatmap(conf_matrix,xticklabels = LABELS,yticklabels = LABELS,annot = True,fmt
="d")
plt.title("Confusion matrix")
plt.ylabel("True class")
plt.xlabel("Predicted class")
plt.show()

#Print Accuracy,Precision and Recall
print("Accuracy :",accuracy_score(error_df['True_class'],error_df['pred']))
print("Recall :",recall_score(error_df['True_class'],error_df['pred']))
print("Precision :",precision_score(error_df['True_class'],error_df['pred']))
```

```
Accuracy : 0.9898704399424177
Recall : 0.8446601941747572
Precision : 0.13425925925925927

C:\Users\AJINKYA\AppData\Local\Temp\ipykernel_18688\2517333832.py:12: User
Warning: FigureCanvasAgg is non-interactive, and thus cannot be shown
  plt.show()
```